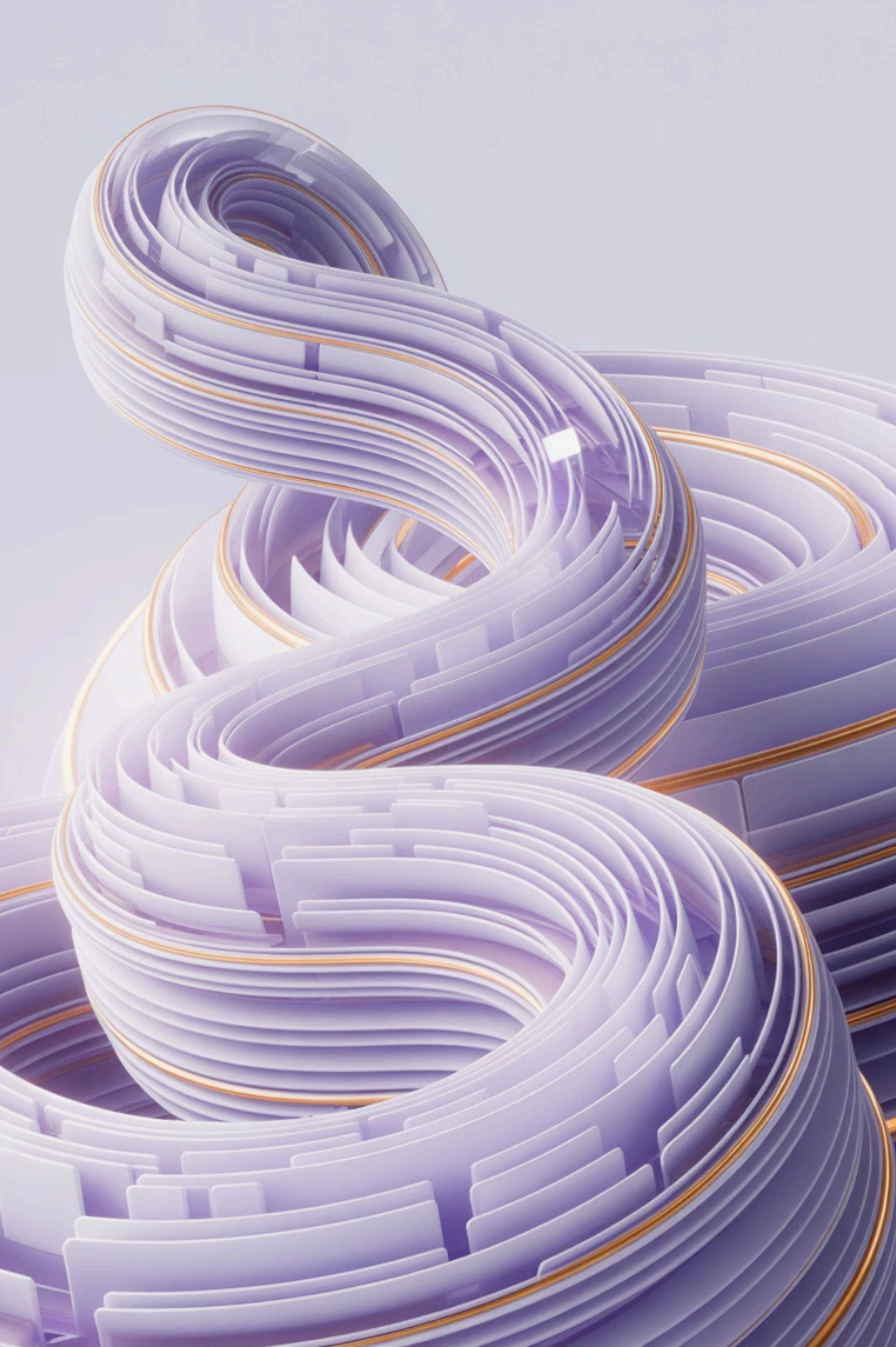




Selectores avanzados y posicionamiento en CSS

Una guía práctica para dominar técnicas de selección precisa y control de posición en diseño web



Combinador descendente (div p)

Los selectores descendentes son fundamentales para apuntar a elementos anidados dentro de otros, sin importar su nivel de profundidad en la estructura HTML.

Explicación

El selector `div p` selecciona **todos** los párrafos que se encuentran dentro de un `div`, independientemente de cuántos niveles de anidamiento existan entre ellos.

Ejemplo de código

```
div p {  
  color: #403ccf;  
}
```

Con este selector, todos los párrafos dentro del `div` se pintarán de color azul, incluso aquellos que están dentro de otros elementos como secciones o artículos anidados.

Combinador hijo directo (div > p)

¿Qué es?

A diferencia del selector descendente, el combinador hijo directo (>) selecciona **únicamente** los elementos que son hijos inmediatos del elemento padre, sin afectar a niveles más profundos.

Este selector es ideal cuando necesitamos aplicar estilos solo al primer nivel de elementos dentro de un contenedor, manteniendo intactos los estilos de elementos más profundos en la estructura.

Ejemplo práctico

```
div > p {  
  color: #ff3333;  
}
```

Pseudo-clase :hover

La pseudo-clase `:hover` permite aplicar estilos cuando el usuario pasa el cursor por encima de un elemento, creando interactividad sin necesidad de JavaScript.



Estado normal

El botón mantiene su apariencia original hasta que el usuario interactúa con él.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
  <title>Hover</title>
</head>
<body>
  <button class="btn">Pasar el ratón</button>
</body>
</html>
```



Estado :hover

Al pasar el cursor, el botón cambia instantáneamente sus propiedades visuales para indicar interactividad.

```
body {
  background-color: #333;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.btn {
  font-size: 18px;
  font-weight: bold;
  padding: 15px 40px;
  border: 2px solid cyan;
  color: white;
  background-color: transparent;
  letter-spacing: 1px;
  cursor: pointer;
  transition: all 0.3s ease-in-out;
  border-radius: 8px;
}

.btn:hover {
  background-color: cyan;
  color: black;
  box-shadow: 0 0 5px cyan, 0 0 10px cyan, 0 0 10px cyan;
  border: none;
}
```

La propiedad `transition` permite que el cambio sea suave y gradual, mejorando la experiencia del usuario.

Pseudo-clase :focus

La pseudo-clase `:focus` se aplica cuando un elemento recibe el foco, generalmente tras hacer clic en él o navegar con el teclado usando la tecla Tab.

Es especialmente útil para campos de formulario, indicando visualmente al usuario qué elemento está actualmente activo.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <link rel="stylesheet" href="style.css">
  <title>Focus</title>
</head>

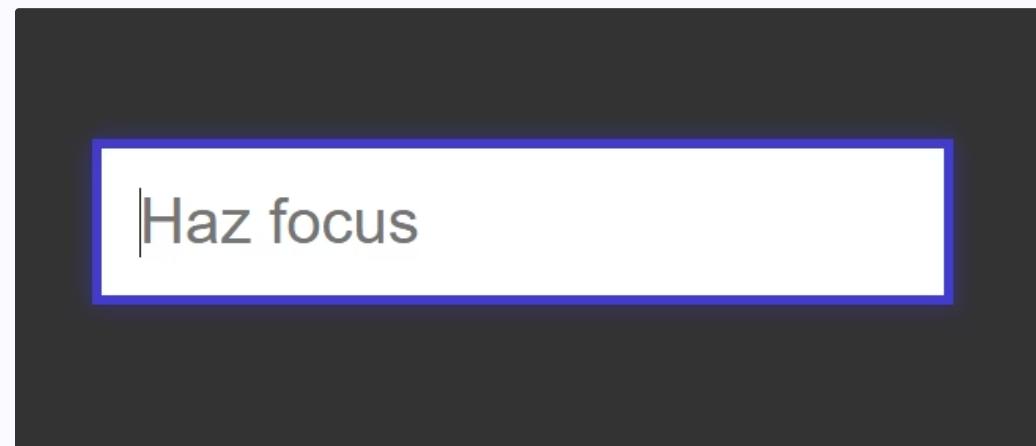
<body>

  <input type="text" placeholder="Haz focus">

</body>

</html>
```

Ejemplo práctico



```
input {
  border: 2px solid #CCCCCC;
  padding: 8px;
  outline: none;
}

input:focus {
  border-color: #403ccf;
  box-shadow: 0 0 5px rgba(64,60,207,0.5);
}
```

- ❏ **Importante para accesibilidad:** Nunca elimines completamente los indicadores de foco, ya que son esenciales para los usuarios que navegan con teclado.

El estilo `:focus` debe ser lo suficientemente distintivo para que el usuario identifique claramente qué elemento está seleccionado, mejorando así la usabilidad del sitio.

Pseudo-clase :nth-child()

Esta potente pseudo-clase permite seleccionar elementos basándose en su posición dentro de su contenedor padre, siguiendo patrones numéricos o fórmulas.

Valores comunes

- `:nth-child(even)` - Selecciona elementos pares
- `:nth-child(odd)` - Selecciona elementos impares
- `:nth-child(3)` - Selecciona el tercer elemento
- `:nth-child(3n)` - Selecciona múltiplos de 3
- `:nth-child(3n+1)` - Fórmulas para patrones complejos

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo nth-child</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <ul>
    <li>Elemento 1</li>
    <li>Elemento 2</li>
    <li>Elemento 3</li>
    <li>Elemento 4</li>
    <li>Elemento 5</li>
  </ul>
</body>
</html>
```

Ejemplo practico



```
ul {
  list-style: none;
  padding: 0;
  width: 300px;
}
li {
  padding: 10px;
  margin: 5px;
  background: #ddd;
}
/* Selecciona el 2do elemento */
li:nth-child(2) {
  background: lightblue;
}
li:nth-child(3) {
  background: blue;
}
/* Selecciona los elementos impares */
li:nth-child(odd) {
  color: red;
}
/* Selecciona los elementos pares */
li:nth-child(even) {
  color: green;
}
```

Esta técnica es perfecta para crear listas con filas alternantes, destacar elementos específicos en secuencias o crear patrones visuales en galerías de imágenes.

Position: static

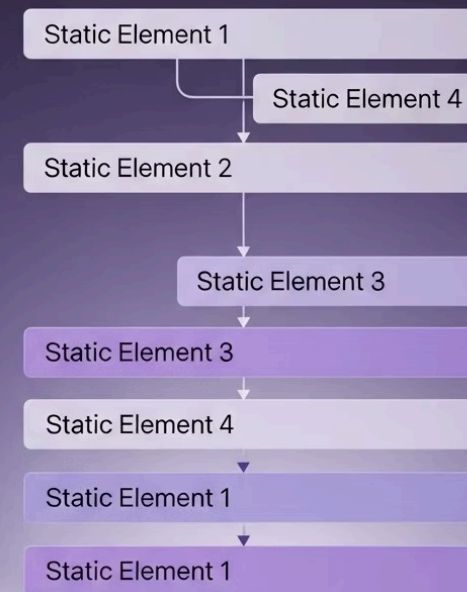
`position: static` es el valor predeterminado para todos los elementos HTML. Significa que el elemento sigue el flujo normal del documento.

Características principales:

- No se ve afectado por las propiedades `top`, `right`, `bottom` o `left`
- Cada elemento aparece debajo del anterior
- El espacio ocupado excluye a otros elementos

```
div {  
  position: static; /* Valor por defecto */  
  width: 200px;  
  height: 100px;  
  background-color: #E7E0FF;  
}
```

CSS Static Positioning Flow



Ejercicio práctico: Crea varios elementos `div` sin especificar ninguna posición y observa cómo se apilan naturalmente uno debajo del otro siguiendo el flujo del documento.

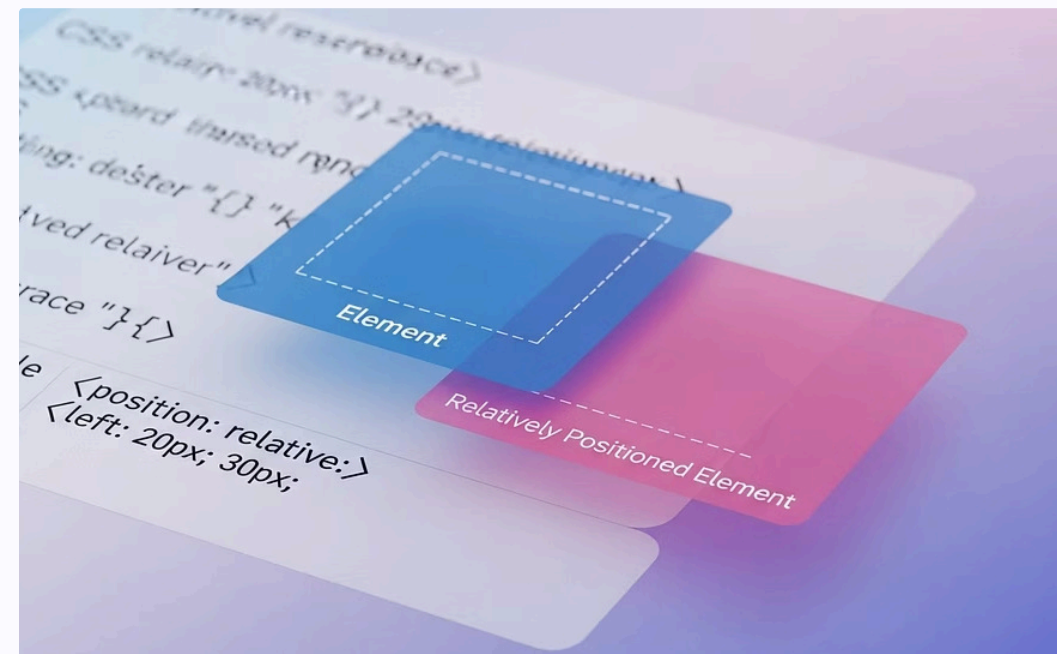
Position: relative

`position: relative` posiciona un elemento en relación a su posición original en el flujo normal del documento.



Desplazamiento

El elemento se desplaza desde su posición original, pero mantiene su espacio reservado en el flujo del documento.



Espacio reservado

Aunque el elemento se mueve, su espacio original permanece ocupado, afectando a la posición de otros elementos.

```
.caja-relativa {  
  position: relative;  
  top: 20px; /* Desplaza 20px hacia abajo */  
  left: 20px; /* Desplaza 20px hacia la derecha */  
  background-color: #403ccf;  
  color: white;  
  padding: 15px;  
}
```

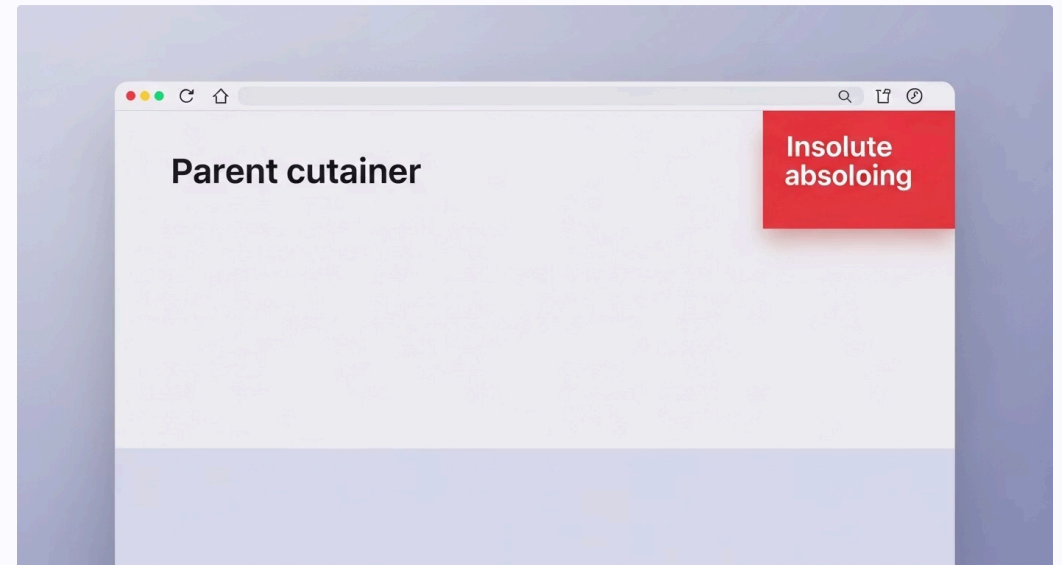
Esta posición es muy útil para ajustes precisos de elementos sin romper el flujo del documento, y también sirve como contenedor de referencia para elementos con `position: absolute`.

Position: absolute

`position: absolute` elimina el elemento del flujo normal del documento y lo posiciona en relación a su ancestro posicionado más cercano.

Características clave:

- Sale completamente del flujo normal
- No reserva espacio en el documento
- Se posiciona respecto al primer ancestro posicionado (con `position` diferente a `static`)
- Si no hay ancestro posicionado, se posiciona respecto al `body`



```
.contenedor {  
  position: relative;  
  width: 300px;  
  height: 200px;  
  border: 1px solid #ccc;  
}  
  
.caja-absoluta {  
  position: absolute;  
  top: 0;  
  right: 0;  
  width: 50px;  
  height: 50px;  
  background-color: #ff3333;  
}
```

Este posicionamiento es ideal para elementos como iconos de notificación, badges, menús desplegables y elementos que deben superponerse a otros.

Position: fixed

`position: fixed` posiciona un elemento en relación a la ventana del navegador, manteniendo su posición incluso cuando se hace scroll.

Características

- El elemento sale completamente del flujo normal del documento
- Permanece fijo en la misma posición aunque se haga scroll
- Se posiciona en relación a la ventana del navegador
- No reserva su espacio original en el layout

Ejemplo de código

```
.menu-fijo {  
    position: fixed;  
    top: 0;  
    left: 0;  
    width: 100%;  
    background-color: #403ccf;  
    color: white;  
    padding: 10px;  
    z-index: 100;  
}  
  
.contenido {  
    margin-top: 60px; /* Espacio para el menú fijo */  
}
```

Este posicionamiento es perfecto para crear barras de navegación que permanezcan visibles durante toda la navegación, botones de "volver arriba" o chats fijos en la esquina inferior.

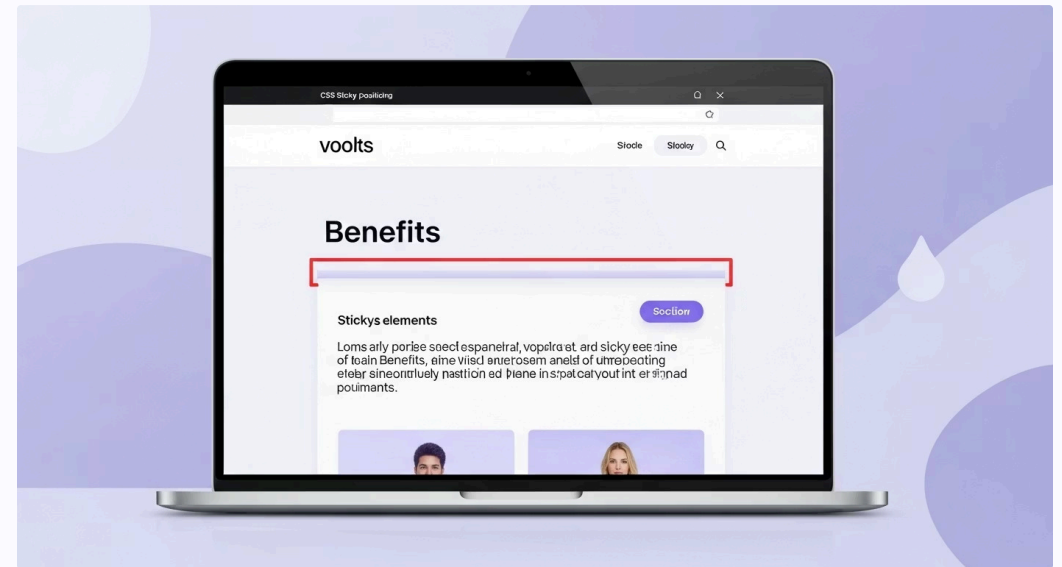
Position: sticky

El posicionamiento `sticky` es un híbrido entre `relative` y `fixed`. El elemento se comporta como `relative` hasta que cruza un punto de desplazamiento especificado, y luego se comporta como `fixed`.

Comportamiento

1. Inicialmente, el elemento se posiciona como `relative`
2. Al hacer scroll y alcanzar el umbral definido, se "pega" como `fixed`
3. Permanece "pegado" hasta que su contenedor padre sale de la vista

❏ Funciona mejor con el posicionamiento `top`, pero también puedes usar `bottom`, `left` o `right` dependiendo de la dirección del scroll.



```
.titulo-sticky {  
  position: sticky;  
  top: 0;  
  padding: 10px;  
  background-color: #E7E0FF;  
  color: #403ccf;  
  z-index: 10;  
}
```

Este posicionamiento es ideal para encabezados de sección, índices de contenido o elementos de filtro que deben permanecer visibles durante la exploración de su contenido relacionado.



Ejercicio Final: Mini Landing Page

Objetivo: Crear una página con HTML y CSS aplicando los siguientes puntos:

1. Estructura HTML

- Un header con menú de navegación fijo (`position: fixed`).
- Una sección principal con:
 - Un título con `position: sticky`.
 - Una lista (`ul li`) donde uses `:nth-child(even)` y `:nth-child(odd)` para alternar estilos.
 - Un botón con efecto `:hover`.
 - Un input con estilo en `:focus`.
- Una sección con varias cajas (`div`) para practicar `position: relative` y `position: absolute`.
- Un footer básico.

2. CSS

- Usa **selectores descendentes** y de **hijo directo**.
- Aplica pseudoclases (`:hover`, `:focus`, `:nth-child()`).
- Experimenta con diferentes posiciones (`static`, `relative`, `absolute`, `fixed`, `sticky`).
- Agrega transiciones para que los cambios sean suaves.

✓ Este ejercicio te permite consolidar los conocimientos sobre el flujo del documento, el impacto de las propiedades de posición y la interactividad con pseudoclases, creando una interfaz web dinámica y funcional.

Código de ejemplo para el ejercicio

Estructura HTML

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini Ejercicio Final</title>
  <link rel="stylesheet" href="styles.css">
</head>

<body>

  <header>
    <h1>Ejercicio Final CSS</h1>
  </header>

  <main>
    <section>
      <button class="btn">Pasa el mouse</button>
      <input type="text" placeholder="Haz focus aquí">
    </section>

    <section>
      <h2>Lista con nth-child</h2>
      <ul class="lista">
        <li>Item 1</li>
        <li>Item 2</li>
        <li>Item 3</li>
        <li>Item 4</li>
      </ul>
    </section>

    <section class="contenedor">
      <p class="caja-relativa">
        Caja relativa
        <span class="caja-absoluta">ABS</span>
      </p>
    </section>
  </main>

  <footer>
    <p>&copy; 2025 - Mini práctica CSS</p>
  </footer>

</body>

</html>
```

Estilos CSS Esenciales

```
/* === ESTILOS GENERALES === */
body {
  font-family: Arial, sans-serif;
  padding: 20px;
}

header, footer {
  text-align: center;
  background: #403ccf;
  color: white;
  padding: 10px;
  margin-bottom: 20px;
}

/* === BOTÓN :HOVER === */
.btn {
  padding: 10px 20px;
  border: 2px solid cyan;
  background: #333;
  color: white;
  cursor: pointer;
  transition: all 0.3s ease;
}

.btn:hover {
  background: cyan;
  color: black;
}

/* === INPUT :FOCUS === */
input {
  padding: 8px;
  margin-left: 10px;
  border: 2px solid #ccc;
  outline: none;
}

input:focus {
  border-color: #403ccf;
  box-shadow: 0 0 5px rgba(64,60,207,0.5);
}

/* === LISTA CON NTH-CHILD === */
.lista {
  list-style: none;
  padding: 0;
}

.lista li {
  padding: 5px;
  margin: 3px 0;
  background: #eee;
}

.lista li:nth-child(odd) { color: red; }
.lista li:nth-child(even) { color: green; }

/* === RELATIVE + ABSOLUTE === */
.contenedor {
  margin-top: 20px;
  border: 2px solid #ccc;
  padding: 20px;
  width: 200px;
}

.caja-relativa {
  position: relative;
  background: #403ccf;
  color: white;
  padding: 20px;
}

.caja-absoluta {
  position: absolute;
  top: 5px;
  right: 5px;
  background: #ff3333;
  padding: 3px 6px;
  font-size: 12px;
  color: white;
}
```

Resultado Final

Ejercicio Final CSS

Pasa el mouse

Haz focus aquí

Lista con nth-child

Item 1

Item 2

Item 3

Item 4

Caja relativa

ABS

© 2025 - Mini práctica CSS

¡Felicidades!

Tu primer web es increíble!